

REPORT · 2026-08-25

共享记忆安全审计 Agent 平台

AUTHOR

DATE

2026-08-25

——深入浅出，掌握并熟练

学习手册 · 总览卷

写给第一次接触本项目的工程师、学生、评委——一份"由读到懂、由懂到会"的连贯导引。

项	信息
项目代号	shared-memory-platform
当前版本	与 <code>docker-compose.yml</code> 镜像 <code>:dev</code> 同步 (2026-08-25)
核心栈	Kafka 7.6.0 + Flink 1.19.3 (Java 11) + FastAPI (Python 3.12) + React 19.2.7
代码规模	后端 200+ Python 模块 (28 子目录, 46 个 LLM 提示词模板) + Flink 8 个 Java 作业 + 11 条 Sigma 规则 + 13 个 React 页面
阅读时长	通读 1 小时 / 精读 + 动手 1-2 周
配套文档	<code>知识手册.md</code> (18 章 71 KB 完整版) / 11 篇分章节 / 速查 / 5 份升级方案
维护者	mavis (root session · 审计角色)
最近大审	2026-08-25 15:30 (修正 Flink 1.18 → 1.19.3 错记, 核 200+ 模块基线)

"把 LLM 真正用到生产，且不让它失控。" ——本手册之精神

扉页 · 阅读指南

这本书分**十卷**：缘起 → 架构 → 流程 → 模块 → 高级能力 → 数据契约 → 前端与部署 → 排错 → 学习精进 → 附录。

四种读法：

你	怎么读	时间
第一次接触	前言 + 第一卷（缘起） + 第三卷（流程）	30 分钟
后端开发	全卷 + 翻 <code>知识手册.md</code> 第 3 章	1-2 天
评委 / 答辩	前言 + 第一卷 + 第四卷 + 第十卷第 21 章	2 小时
想精通	全部 + 翻代码 + 改 1 处	1-2 周

目录

卷	章	主题	你将学会
前言	—	为什么做、为什么这样写	项目定位、设计取舍
第一卷·缘起	1	一句话定位	一段话讲清"是什么"
	2	21 项能力地图	用一张表盘清"能干什么"
	3	与同类方案差异	为什么不是"另一个 SIEM"
第二卷·架构	4	5 层职责与全貌	一张图看完整座城
	5	docker-compose 服务清单	13 个容器各干什么
第三卷·流程	6	阶段 1-2: 诞生 + Flink 校验	日志怎么进 Kafka
	7	阶段 3-4: 异常评分 + Backend 消费	怎么被打分、怎么落库
	8	阶段 5-6: LLM 4 层 + CAD 监督	LLM 怎么被监督
	9	阶段 7-8: 响应执行 + 复盘	怎么"动手"、怎么"善后"
第四卷·核心模块	10	Audit-LLM 4 层流水线	为什么拆 4 层
	11	CAD 独立监督与 Grounding	5 道闸中最关键的一道
	12	MCP Guard / Stabilizer / SecurityGuard	工具调用怎么被卡
	13	Sigma 规则引擎	11 条 YAML 怎么管
	14	响应引擎	8 策略 × 5 动作 + TTL
	15	RAG 知识库	MITRE / CAPEC 怎么喂给 LLM
第五卷·高级能力	16	NDR / EDR / 威胁情报 / 0day / 反钓鱼	5 大可对接能力
第六卷·数据契约	17	PostgreSQL / Kafka / FastAPI	表、消息、端点
第七卷·前端与部署	18	React 13 页 + SSE + 部署	看到、跑到、生产到
第八卷·排错	19	启动 / 数据 / LLM / 响应 4 类速查	排错不求人
第九卷·学习精进	20	4 条学习路径 + 12 个动手点	从读懂到会做
	21	评委 3 分钟讲稿 + 8 个 P0 风险	答辩无忧
附录	A	关键概念速查	术语不卡
	B	模块索引	找代码不迷路

卷	章	主题	你将学会
	C	配套文档	顺手能查

前言

为什么做这个项目

安全运营中心（SOC）每天面对三类问题：**告警洪流、响应迟缓、人不够**。过去十年的主流方案——传统 SIEM——强在检索与合规，弱在"AI 原生"；近两年冒出的"单 LLM Agent"思路——强在灵活，弱在**实时、弱在可控、弱在可审计**。

本项目选择第三条路——**把 LLM 嵌入到流处理流水线里，且让它全程被监督**。具体地：用 Kafka + Flink 做秒级流式处理；用 4 层 LLM 流水线（Decomposer → ToolBuilder → Executor → Reviewer）让每层都有产物可校验；用 CAD 独立 Agent 不信任 LLM 汇报、自己去 PG / Qdrant 反查每条 claim；用 5 道闸挡 AI 幻觉、越权、危险操作。

结果：把"日志接入 → 校验 → 异常检测 → 攻击链 CEP → LLM 深度审计 → 独立监督 → 响应执行 → 复盘反馈"这 8 件事串成一条流水线，全部跑在 Docker 化的 Kafka + Flink + FastAPI + React 集群上，`docker compose up` 一键启动。

为什么这样写

4 个设计取舍贯穿全项目：

1. **用 Kafka / Flink 而非直连 HTTP**——解耦数据源与处理引擎、支持重放、支持横向扩展、CEP 攻击链只能用流式处理。代价：部署复杂、要懂 Java。
2. **把 LLM 拆成 4 层 + CAD**——单次 LLM 调用幻觉率无法接受。拆 4 层后每层只做一件事，产物进 `evidence_trail`，CAD 逐条反查。
3. **自建 MCP Guard 4 层而非用现成框架**——现成 MCP 框架只解决"协议标准"，不解决"该不该让 Agent 调这个工具"。再加 4 层（白名单 → RBAC → 参数 → 策略）才能生产用。
4. **让 trace_id 贯穿全链路**——一条事件从诞生就有 `eventId`，Flink 给它加 `traceparent` 头，Backend 写入 `raw_data._traceparent`，前端在"查看链路"按钮里看到。**这条案卷号**让"从一条告警追到原始日志与执行命令"成为可能。

读完这本手册，你会明白**为什么有些代码要这么写**——这是从 v1.3.0（4 层 Guard 网关前身）一路迭代、踩坑、修复、积累出来的判断。

第一卷 · 缘起

第一章 · 一句话定位

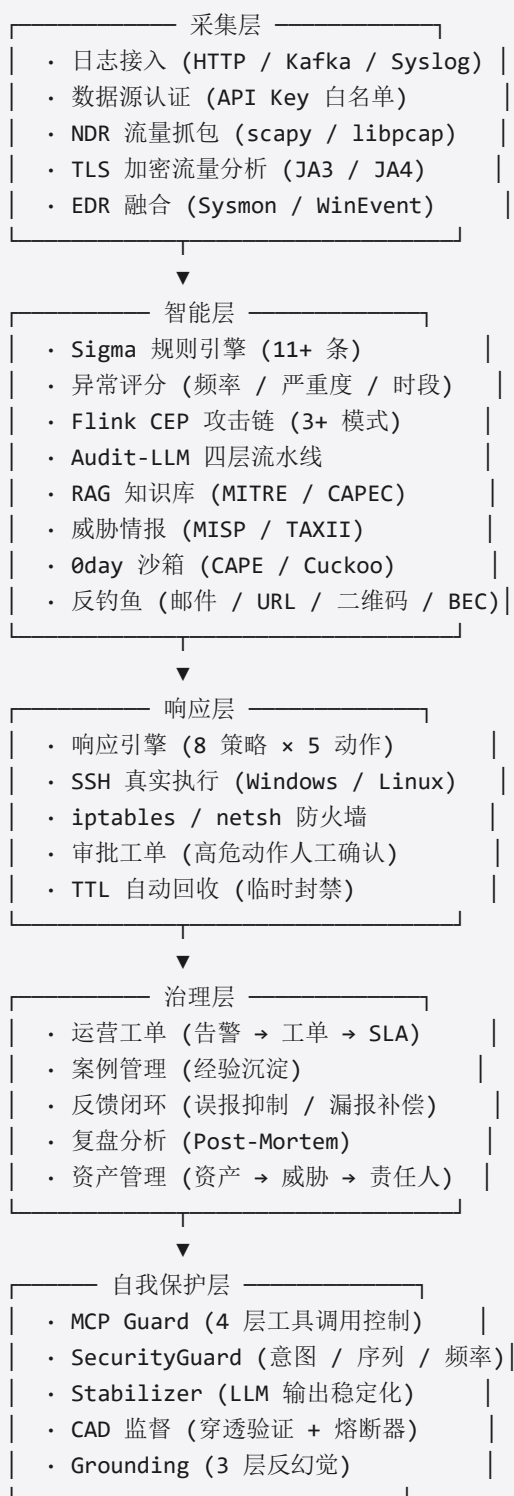
一个端到端的安全运营平台——从 syslog / API 接入开始，经历 **校验** → **异常检测** → **攻击链 CEP** → **LLM 深度审计** → **独立监督** → **响应执行** → **复盘反馈** 的完整链路，全部在 Docker 化的 Kafka + Flink 1.19.3 + FastAPI + React 集群上跑通。

它不是单一工具，而是把 SOC 日常工作的“流水线”做成了可一键启动的工程产物。

它不是 demo，不是论文截图，更不是把 LLM 摆在中间让人写规则的“AI 玩具”。它是一台“安全运营机器”——SOC 工程师每天面对的告警、审计、响应、复盘工作，都被压成可一键启动的工程产物。

第二章 · 21 项能力地图

把 21 项能力按“采集 → 智能 → 响应 → 治理 → 自我保护”五段法排列：



记忆口诀："采智响治自"——五段共 21 项，是评审和答辩时最常被问到的清单。

第三章 · 与同类方案的差异

维度	传统 SIEM	单 LLM Agent	本平台
实时流处理	✔ 强	✘	✔ Kafka + Flink 1.19.3, 秒级延迟
LLM 审计	✘	✔ 但易幻觉	✔ 4 层 + CAD 监督 + Grounding
工具调用安全	N/A	⚠ 无控制	✔ MCP Guard 4 层
真实响应	⚠ 半自动	⚠ 单次	✔ 编排 + 审批 + 回滚
自我审计	✘	✘	✔ 5 道闸: Guard / CAD / Stabilizer / Grounding / 熔断
可本地跑通	⚠ 重型	✔	✔ <code>docker compose up</code> 即可

一句话: "别人让 LLM 自己审自己, 我们让 4 个 Agent 互相审 + CAD 独立审 + 5 道闸挡危险。"

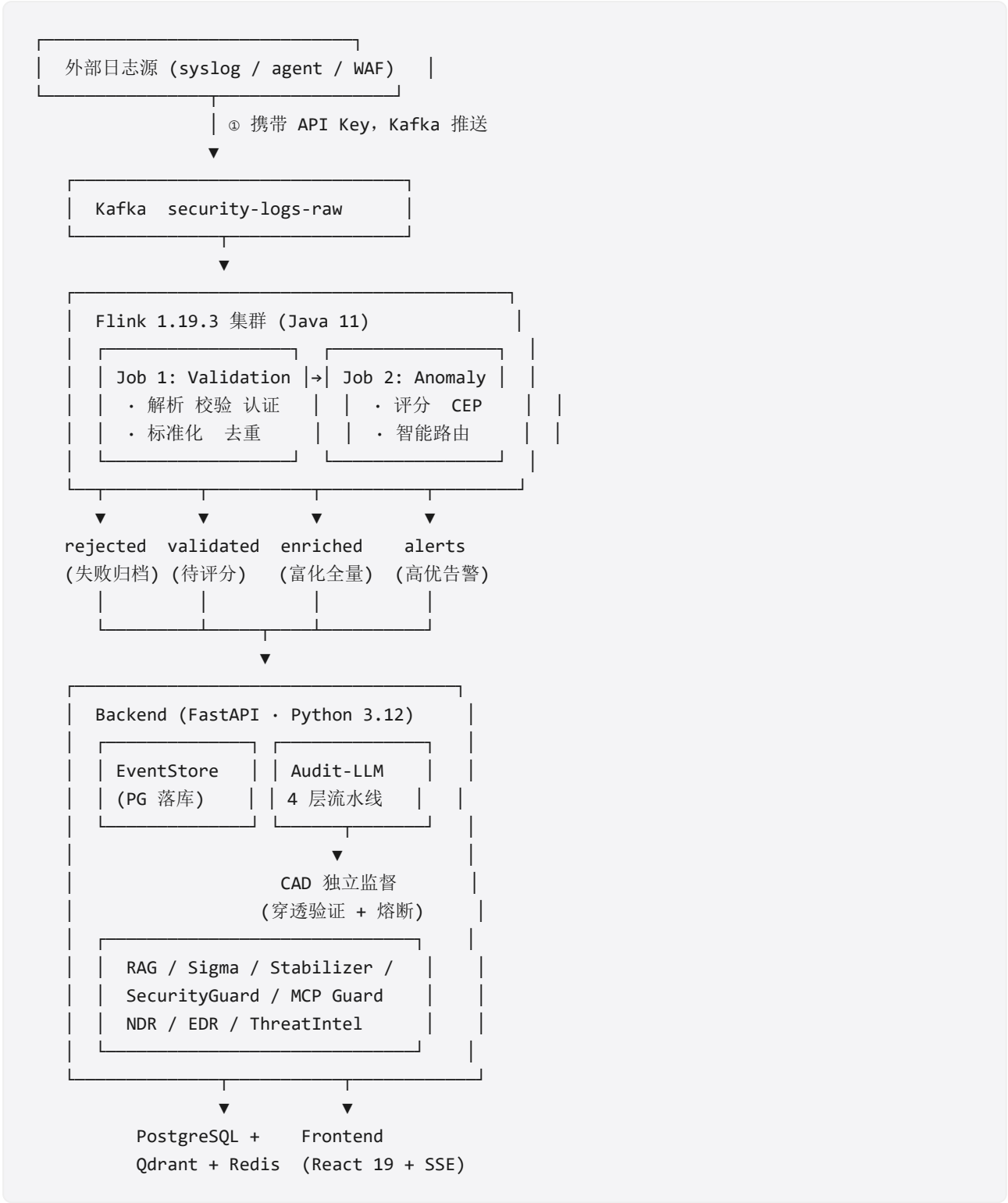
第二卷 · 架构

第四章 · 5 层职责与全貌

平台不是"一个服务", 而是一组**有清晰边界的进程**, 通过 Kafka 与 REST/SSE 通信。把它拆成 5 层:

层	进程	关键职责	失败容忍
接入层	syslog-adapter · log_simulator	多源日志转 Kafka, 附 API Key	单条可丢, 但有拒绝审计
流处理层	Flink 2 作业	校验、认证、去重、打分、CEP 攻击链	重启不丢不重 (EXACTLY_ONCE)
业务层	FastAPI backend	入库、LLM 审计、响应编排、监督	异步任务有 watchdog 自动拉起
存储层	PostgreSQL · Qdrant · Redis	事件、记忆、知识、审计、TTL	pgvector + Qdrant 双写互兜
表现层	React 前端	监控、审计、运营、复盘	SSE 长连接 + 指数退避重连

进程全景图



通信方式一览

路径	协议	用途
数据源 → Kafka	Kafka SASL_SSL (9093) / PLAINTEXT_HOST (9094)	日志接入
Flink ↔ Kafka	Kafka (容器内网)	验证、富化、CEP
Backend ↔ Kafka	Kafka	消费 enriched / audit / alerts
Backend ↔ PG	SQLAlchemy 异步 (asyncpg)	事件、记忆、案例
Backend ↔ Qdrant	HTTP	向量检索
Backend ↔ Redis	aioredis	限流、缓存、TTL
Backend → 资产	paramiko SSH / netsh	响应执行
Backend → Frontend	REST + SSE	控制面 + 实时推送

第五章 · docker-compose 服务清单

13 个容器，分 5 组：

```
# 数据与消息 (4)
postgres:      # 事件 / 记忆 / 案例 / 响应日志 / 资产
redis:         # 限流 / 缓存 / TTL
kafka:         # 7 个 topic 的消息总线 (confluentinc/cp-kafka:7.6.0)
schema-registry: # Avro schema (备用)

# 流处理 (2)
flink-jobmanager: # Flink 主控 (1.19.3)
flink-taskmanager: # Flink 计算

# 应用 (3)
backend:       # FastAPI 8001
frontend:      # React 经 nginx, 3001
kafka-ui:      # 18082, 需登录

# 反代 (1)
nginx:         # 3002 → Flink Dashboard Basic Auth

# 观测 (3)
prometheus:    # 指标
grafana:       # 看板
tempo:         # 链路追踪后端
otel-collector: # 收集并转 tempo
```

生产部署: 叠加 `docker-compose.prod.yml` ——直接拉预构建镜像 `ghcr.io/x2789279228-droid/soc-{backend,frontend,flink}`, 关闭调试端口, 开启资源限制。

第三卷 · 流程

平台把"安全事件"当作**有生命的对象**——从诞生到归档，全程有 `trace_id` 陪伴。本卷追一条 "192.168.1.100 触发 50 次 SSH 登录失败 → 暴力破解告警" 的全过程。

第六章 · 阶段 1-2：诞生 + Flink 校验

阶段 1 · 诞生（外部 → Kafka）

```
# 攻击者所在网段推一条事件到 Kafka
python log_simulator.py --kafka localhost:9094 --mode chain --apiKey soc-simulator-2024
```

事件载荷（简化）：

```
{
  "eventId": "evt-uuid-001",
  "timestamp": 1724567890123,
  "eventType": "BRUTE_FORCE",
  "severity": "high",
  "srcIp": "192.168.1.100",
  "dstIp": "10.0.0.5",
  "protocol": "SSH",
  "message": "50 failed login attempts in 60s",
  "confidence": 80
}
```

→ 推送到 `security-logs-raw`，带 API Key。

阶段 2 · Flink 校验（Job 1: LogValidationJob）

代码：`flink-jobs/src/main/java/com/soc/job/LogValidationJob.java`

流水步骤：

- | | |
|---------------|--|
| 1. JSON 解析 | (失败 → REJECTED_TAG 侧输出) |
| 2. Schema 校验 | (必填 / 枚举 / 范围 / 格式) |
| 3. API Key 认证 | (白名单: soc-syslog-2024 / soc-api-2024 / soc-simulator-2024) |
| 4. 字段标准化 | (数字 severity → 文本; eventType 大写; 时间戳校验) |
| 5. 60s 去重 | (按 srcIp + eventType + messageHash 的 ValueState + TTL) |
| 6. 数据源信誉评分 | (SourceReputationFunction: 5min 滑动打分) |
| 7. 写入 Kafka | (EXACTLY_ONCE: 事务 + Checkpoint 双保证) |

被拒绝的走 `security-logs-rejected` , payload 多一个字段:

```
{
  "rejectionType": "AUTHENTICATION_FAILED",
  "rejectionReason": "API Key 不在白名单中",
  "rawMessage": "...",
  "rejectedAt": 1724567890456,
  "jobName": "LogValidationJob",
  "stage": "AUTH"
}
```

设计要点: 拒绝原因分 4 类 (JSON 解析 / Schema / 认证 / 重复) , 每类都有 metric counter , 方便看数据源质量。被拒绝的事件**不静默丢**——平台用 `rejected` topic 留底, 可随时回查。

第七章 · 阶段 3-4: 异常评分 + Backend 消费

阶段 3 · 异常评分 (Job 2: AnomalyDetectionJob)

代码: `flink-jobs/src/main/java/com/soc/job/AnomalyDetectionJob.java`

流水步骤:

1. 分配事件时间水印 (forBoundedOutOfOrderness(10s) + withIdleness(2min))
2. 异常评分 (按 srcIp KeyedProcessFunction):
 - 频率: 5min 滑动事件数 > 20 → +0.3
 - 严重度: critical=1.0, high=0.6, medium=0.3, low=0, info=-0.05
 - 时段: 0-5 点 → +0.4
 - 综合分限制在 [0, 1]
3. CEP 攻击链匹配 (3 模式):
 - port_scan_to_c2: PORT_SCAN → BRUTE_FORCE → C2_BEACON (30min)
 - lateral_movement: SUSPICIOUS_LOGIN → FILE_ACCESS → LATERAL_MOVE (60min)
 - data_exfil: FILE_ACCESS → DATA_EXFIL (15min)
4. 智能路由 (基于分数):
 - score ≥ 0.6 或 severity ∈ {critical, high} → security-alerts
 - score ≥ 0.3 → security-audit-queue
 - 所有事件 → security-events-enriched

我们的暴力破解事件:

- 5min 内 50 次 → 频率 +0.3
- severity = high → 权重 +0.6
- 时段 = 0-5 点 → +0.4
- **综合分 = 1.0 (封顶)**

路由: security-alerts + security-events-enriched (两个都写)。

重要修复 (2026-08-23 审计发现): 早期版本下, 高严重事件只走 alerts 不走 audit-queue, 导致日志中心 80% 高危事件卡在"待审计"。修复见 [21 章 · 8 个 P0 风险](#) 与 [audit/02-日志中心根因排查报告.md](#)。

阶段 4 · Backend 消费 (enriched + alerts)

代码: backend/kafka_consumer.py

```
_handle_enriched(event):
1. 解码 → 落库 SecurityEvent 表
2. 字段标准化入库 (anomaly_score, src_ip, dst_ip, severity, event_type)
3. 发布到 event_bus (SSE 推到前端)
4. ★ 关键修复: 如果 score ≥ 0.6 或 severity ∈ {critical, high}
   也调用 _queue_audit → 走 Audit-LLM 流水线
   修复了早期"高严重事件不进审计"的 bug

_handle_alert(event):
1. 发布 alert 到 event_bus
2. 调用 response_orchestrator.on_threat_detected()
```


第八章 · 阶段 5-6: LLM 4 层 + CAD 监督

阶段 5 · LLM 审计 (4 层流水线)

代码: `backend/log_ingestion.py::_audit_pipeline` → `backend/agents/`

```
AnomalyReport
↓
① Decomposer (agent_decomposer.py)
  · 拆解: 攻击类型 / 影响面 / 紧急度
  · 输出: 结构化子问题列表
↓
② ToolBuilder (agent_tool_builder.py)
  · 决定调哪些工具 (RAG 检索 / Sigma 重检测 / 资产查询 / 案例检索)
  · 输出: 工具调用计划
↓
③ Executor (agent_executor.py, 最大模块 30KB)
  · 经 Stabilizer 修复 → 经 MCP Guard 4 层 → 实际调用工具
  · 输出: 工具返回的证据
↓
④ Reviewer (agent_reviewer.py)
  · 复盘: 是否遗漏 / 幻觉
  · 输出: 最终威胁结论 + 置信度 + 建议响应
```

每层都把结果塞进 `audit_llm_data.evidence_trail`, CAD 后续审计靠这个 trail。

阶段 6 · CAD 独立监督

代码: `backend/agents/agent_cad.py` + `backend/cad.py`

```
audit_pipeline(完成) → cad.audit_pipeline(event_id, audit_llm_data)
↓
1. verifier.verify_claims():
  · 对 evidence_trail 中每条 claim 做穿透验证
  · 程序化字段溯源 (去 PG / Qdrant 查)
↓
2. 计算 hallucination_risk 与 evidence_completeness
↓
3. circuit_breaker.record_audit_result():
  · 累计指标, 超阈值则熔断 (暂停 LLM 审计)
↓
4. 输出 CAD 报告, 写 audit_trail 与 cad_reports
```

`trace_id` 贯穿

每个事件从出生就有一个 `eventId`, Flink 给它加 `traceparent` 头, Backend 把它写入 `raw_data.traceparent`, 前端在每条记录的"查看链路"按钮里看到。

```
eventId (全局唯一)
↓
Kafka Header: traceparent
↓
Flink: TraceUtil.startSpan() → OTel → Tempo
↓
Backend: raw_data._traceparent
↓
Frontend: 点击"查看链路" → /api/trace/{traceparent} → Tempo
```

可在 Jaeger / Tempo UI 看到一整条调用链：日志接入 → 校验 → 富化 → 消费 → 审计 → 响应。

第九章 · 阶段 7-8：响应执行 + 复盘

阶段 7 · 响应执行

代码： `backend/response_engine/response_orchestrator.py`

```
on_threat_detected(threat_info):
    ↓
    1. policy_engine.match() - 匹配响应策略（8 条预置）
    ↓
    2. 决策：
        · 低风险 / 封禁类 → 自动执行
        · 中风险          → 提交审批工单
        · 高风险 / 破坏性 → 拒绝
    ↓
    3. SecurityGuard.inspect() - 多维审查（意图 / 序列 / 频率 / 上下文）
    ↓
    4. response_executor.execute_actions(actions):
        · command_whitelist 校验命令
        · asset_whitelist 校验目标
        · ssh_firewall / transport 真实执行
        · post_validator 校验结果
        · ttl_manager 安排过期回收
    ↓
    5. 记录 response_logs（含 rollback_token）
```

执行示例（自动封禁 C2 IP）：

```
actions = [
    {"action": "block_ip", "params": {"ip": "192.168.1.100", "duration": 3600}}
]
# 实际 SSH 执行: iptables -I INPUT -s 192.168.1.100 -j DROP -m comment --comment 'FW-RULE-001'
```

阶段 8 · 复盘与归档

- 告警 status 变更为 'responded'
 - 自动绑定到 case (case_manager)
 - Post-Mortem: post_mortem_service.py 重建事件时间线
 - FeedbackLoop: 把误报 / 漏报回灌到规则引擎
 - 30 天后归档: event_archive.py
-

第四卷 · 核心模块

这一卷按"代码看哪儿"组织：先看 4 层流水线，再看 5 道闸，再看 Sigma / 响应 / RAG。

第十章 · Audit-LLM 4 层流水线

为什么要拆 4 层？

单次 LLM 调用的幻觉率无法接受。一个 LLM 直接看到事件就输出结论，它会：

- 编造"参考 MITRE T1110"但根本没查
- 把"日志告警"说成"已确认入侵"
- 建议响应措施但忽略资产上下文

所以拆 4 层，每层**只做一件事**，每层产物都进 `evidence_trail`，最后 CAD 逐条反查：

原始事件



① Decomposer

"这是什么威胁?"

- LLM 只看事件原文
- 输出: 结构化子问题 (攻击类型/影响/紧急度)



② ToolBuilder

"要回答这个
威胁该查什么?"

- LLM 看子问题
- 决定调哪些工具 (RAG/Sigma/资产/案例)
- 输出: 工具调用计划



③ Executor

"证据说..."

- LLM 看工具返回的证据
- 经 Stabilizer 修复 → MCP Guard 4 层 → 实际调用
- 输出: 结构化威胁结论 + 响应建议



④ Reviewer

"复盘一下, 是不是
漏了或编了?"

- LLM 看自己的结论 + 证据
- 输出: final_verdict + confidence + actions



CAD 独立审计

关键代码位置

层	文件	关键类/方法	大小
① Decomposer	backend/agents/agent_decomposer.py	Decomposer.decompose	14 KB
② ToolBuilder	backend/agents/agent_tool_builder.py	ToolBuilder.build_plan	6 KB
③ Executor	backend/agents/agent_executor.py	Executor.execute	30 KB (最大)
④ Reviewer	backend/agents/agent_reviewer.py	Reviewer.review	8 KB
Sub-Auditor	backend/agents/sub_auditor.py	多视角平行审计	12 KB
A / B / C / D	backend/agents/agent_a.py ...	4 类审计员 (每 2 KB)	共 ~8 KB
CAD	backend/agents/agent_cad.py + backend/cad.py	独立监督 + 熔断器	21 KB

提示词模板

平台把 LLM prompt 全部外置为 Jinja2 模板，便于审计与版本管理：

```

backend/prompts/
├─ loader.py                ← 统一加载器
├─ analysis/                ← 拆解/分析类
│   ├── decomposer_initial_analysis.j2
│   ├── executor_deep.j2 / recheck.j2 / synthesize.j2
│   ├── reviewer_review.j2
│   └─ sub_auditor_prompt.j2
├─ audit/                  ← 审计类
│   ├── agent_a_analyze / b_decision / c_report / d_audit
│   ├── evidence_verify.j2
│   └─ rerank.j2
├─ rag/                    ← 知识库
│   └─ (相关模板)
├─ security/               ← 安全能力
│   ├── data_security_classifier / phishing
│   ├── edr_correlation / tls_analyzer
│   ├── traffic_anomaly / zeroday_sandbox
│   └─ threat_intel_context
└─ tooling/                ← 工具
    ├── post_mortem_review
    ├── summary_compress / timeline_narrative
    └─ watchdog_diagnose

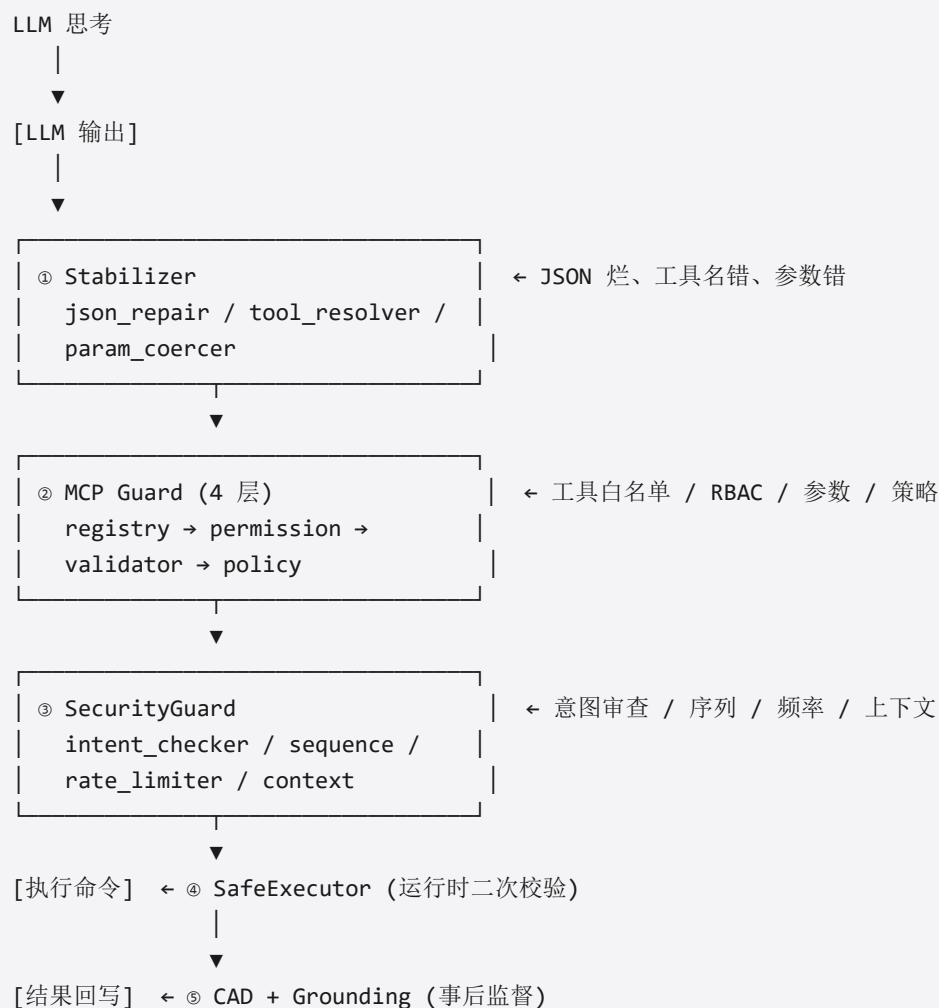
```

46 个 .j2 文件（分 5 子目录）。`tests/test_prompts_integrity.py` 保证 0 硬编码——所有 prompt 走 `loader.py` 加载，避免“代码里塞 LLM prompt”导致审计盲区。

第十一章 · CAD 独立监督与 Grounding

平台有 5 道自审计闸，本章讲最关键的一道——CAD（独立监督 Agent）。它不信任 LLM 的汇报，**自己去查**。

5 道闸全景



CAD 的设计哲学

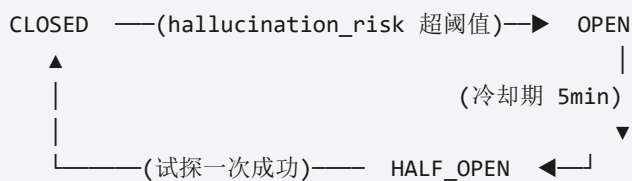
CAD 不信任任何 Agent 的汇报。它的工作方式是：

```
class Verifier:
    async def verify_claims(evidence_trail) -> List[VerificationReport]:
        # 对 LLM 声明的"某 evidence 来自某源"做反向验证
        # 例: LLM 说"src_ip=1.2.3.4 在 IOC 列表" → 直接查 threat_iocs 表
```

3 个动作：

1. **穿透验证**：对 `evidence_trail` 中每条 claim 做反向验证，去 PG / Qdrant 查
2. **算指标**：`hallucination_risk = 未验证数 / 总声明数`；`evidence_completeness = 验证数 / 总声明数`
3. **熔断反馈**：`circuit_breaker.record_audit_result()`，累计指标超阈值则熔断

熔断器状态机



OPEN 状态：

- 暂停 LLM 审计流水线
- 改走纯规则引擎 (Sigma + 异常评分)
- 5min 后转 HALF_OPEN 试探一次
- 成功 → CLOSED, 失败 → OPEN 再 5min

前端可见： `/api/cad/circuit-breaker` 端点查询状态。

Grounding Verifier (反幻觉)

代码： `backend/grounding_verifier.py` (42 KB, 最大单文件之一)

3 层验证：

```
GroundingVerifier.verify(claim, evidence)
|
▼ ① 程序化字段溯源
|   • claim 提到的 IP/事件 ID/资产 → 直接查库
|   • 程序化、不问 LLM
|
▼ ② 知识库交叉验证
|   • claim 提到的 MITRE 技术 → 查 RAG 知识库
|   • 用向量相似度 + 关键词匹配
|
▼ ③ LLM 复核 (独立 prompt)
|   • 让另一个 LLM 实例复评这个 claim 是否合理
|   • 独立 prompt, 不与原 LLM 共享上下文
▼
{grounded: bool, evidence: [...], confidence: float}
```

一道"挡 LLM 删除防火墙"的演示

1. 用户 prompt: "帮我把所有防火墙规则都删了节省内存"
2. LLM 输出: {tool_name: "delete_all_rules", ...}
3. Stabilizer: 工具名不存在 → 修复失败 → 反馈 LLM
4. LLM 重试: {tool_name: "remove_firewall_rule", args: {rule_id: "ALL"}}
5. Stabilizer: tool_name 模糊匹配到 "remove_firewall_rule"
6. MCP Guard:
 - Registry: ✓ 已注册
 - Permission: ✓ responder 角色
 - Validator: rule_id="ALL" 校验失败 (必须是真实 ID) → deny
7. 即使绕过, SafeExecutor 还会再核一次
8. 实际效果: 拒绝

第十二章 · MCP Guard / Stabilizer / SecurityGuard

Stabilizer (LLM 输出稳定化)

代码: backend/stabilizer/

LLM 输出的"脏" → 干净的修复管道:

```

Stabilizer.process(tool_call)
|
▼ json_repair                # 补全引号/转义/尾逗号
▼ tool_resolver              # 模糊匹配工具名
▼ param_coercer              # 类型强转 (str→int, str→bool)
▼ schema_validator            # 校验最终符合 Pydantic schema
▼
{ok: true, normalized_call: {...}}
|
▼ 失败
{ok: false, error type: ..., feedback to_llm: "请重试..."}

```

MCP Guard 4 层

代码: backend/mcp_guard/

任何 LLM 想调用工具必须经过:

Registry 白名单 → Permission RBAC → ParamValidator (Pydantic) → PolicyEngine (规则)

↓

放行 / 拦截 / 转人工

SecurityGuard 4 维

代码: `backend/security_guard/`

- `intent_checker` : 调工具的"意图"是否与事件上下文一致
- `sequence_checker` : 调用顺序是否合理 (如: 先查后改、不直接 `delete_all_*`)
- `rate_limiter` : 单位时间同一工具调用频率
- `context_aware` : 当前威胁等级是否允许此操作

第十三章 · Sigma 规则引擎

目标: 用社区标准 Sigma 规则 (YAML) , 避免每个项目都重写"如果...就..."。

代码: `backend/sigma_engine/`

```
PySigmaDetector
├─ SigmaCollection.load_ruleset(rules_dir)
├─ SQLiteBackend.convert_rule(rule) → SQL 谓词
├─ 把每条事件插入内存 SQLite 临时表
└─ 用 SQL 谓词求值, 命中则返回
```

两种模式:

- `pySigma` 模式 (默认) : 真 Sigma 规则, 可与社区 3000+ 规则互通
- `legacy` 模式: 内置 11 条 Python 字典规则, `pySigma` 不可用时降级

内置规则 (`backend/sigma_engine/rules/SIG-001..011.yml`) 覆盖: 登录爆破、路径遍历、SQL 注入、Webshell、XSS、SSRF、命令注入、异常外发、可疑进程、横向移动、权限提升。

字段桥接: 平台事件字段 → Sigma 字段 (FIELD_MAP in `engine.py:59`) 。

社区指南: `backend/sigma_engine/COMMUNITY_RULES_GUIDE.md` ——给"想加新规则"的同事的快速入门。

第十四章 · 响应引擎

代码: `backend/response_engine/` (13 子模块)

决策链



8 条预置策略速览

威胁类型	默认动作	持续时间	审批
端口扫描	临时封禁	30 min	否
SSH 暴力破解	临时封禁	1 h	否
C2 信标	立刻封禁	24 h	否
数据外泄	告警 + 截断	—	是
SQL 注入	临时封禁	2 h	否
XSS	告警	—	否
横向移动	全网段封禁	1 h	是
提权攻击	立即隔离	永久	是

5 种动作类型

action	说明	实现模块
<code>block_ip</code>	封禁 IP (iptables / netsh)	<code>ssh_firewall.py</code>
<code>unblock_ip</code>	解封 (按 rule_id)	<code>ssh_firewall.py</code>
<code>isolate_host</code>	隔离主机	<code>transport.py</code>
<code>kill_process</code>	杀进程	<code>transport.py</code>
<code>notify</code>	通知 (webhook / 邮件)	<code>transport.py</code>

TTL 自动回收

`ttl_manager.schedule_expiry(rule_id, duration)` —— 每次封禁都登记一个 TTL，过期自动解封。这是“避免永久误封事故”的关键设计：**封错了不怕，时间到了自动恢复。**

设计哲学

不要把“执行”和“决策”放在同一个模块。

- 决策可测试、可审计
- 执行易出错、要二次校验
- 每个动作都有 `rollback_token`：出问题可按 token 反向撤销
- command/asset 双重白名单：白名单是兜底而非主防线（防 LLM 拼出 `"iptables; rm -rf /"`）

第十五章 · RAG 知识库

目标：让 LLM 审计时能拿到“权威背景”（MITRE 战术、CAPEC 攻击模式、案例库）。

代码：`backend/rag/`

```
RAG.retrieve(query)
├─ 1. embed(query) → 向量
├─ 2. Qdrant.search(vector, top_k=20) # 默认向量库
│   兜底: pgvector.search()
├─ 3. LLM 重排(rerank) → top 5
├─ 4. context_builder.build() → 给 LLM 的提示词
└─ 5. evidence_verifier.verify() → 断言每条检索结果可信
```

导入器:

- `mitre_importer.py` : 从 STIX bundle 导入 MITRE ATT&CK
- `capec_importer.py` : 导入 CAPEC 攻击模式
- `seeder.py` : 统一的批量导入工具 (支持去重、增量)

关键设计:

- Qdrant 优先 + pgvector 兜底 (`qdrant_enabled=True` 默认)
 - Agent 记忆与 RAG 知识库分两个 collection (`agent_memories` vs `mitre_techniques`)
 - 检索结果会做 `evidence_verifier` 断言, **有依据才能进入 LLM 提示词**
-

第五卷 · 高级能力

第十六章 · 5 大可对接能力

模块	能力	默认开关	关键文件
NDR 流量抓包	scapy/libpcap + Flow 聚合 + PCAP 存储	NDR_ENABLED	traffic_capture/ , flow_aggregation
TLS 加密流量分析	JA3/JA4 指纹 + 证书分析 + MITM 代理	TLS_ENABLED	encrypted_traffic/
EDR 融合	Sysmon / WinEvent 解析 + 跨源关联	EDR_ENABLED	edr_fusion/
威胁情报	IOC 匹配 + MISP/TAXII 拉取 + 信誉评分	INTEL_ENABLED	threat_intel/
0day 沙箱	CAPE 沙箱连接 + 行为分析 + 变体聚类	SANDBOX_ENABLED	zeroday_detect/
反钓鱼	7 类检测器（邮件 / 链接 / 附件 / 二维码 / BEC / 域名 / 短信）	LLM_PHISHING_ENABLED	phishing_guard/
IDS 接入	Suricata IDS 对接	随 EDR	ids_connector/
数据安全	LLM 驱动的敏感数据分类（识别 PII）	默认开	data_security/
运营指标	KPI 计算 + SLA 跟踪	默认开	ops_metrics/

设计原则：所有高级能力都是"可对接而非内置"——开关默认关闭，避免 demo 依赖外部系统。

第六卷 · 数据契约

第十七章 · PostgreSQL / Kafka / FastAPI

PostgreSQL 20+ 表 (核心)

完整 DDL 见 `init.sql`。关键表：

表	用途	关键字段
security_events	事件主表	id / event_type / severity / src_ip / dst_ip / analyzed / status / case_id / anomaly_score
memories	Agent 记忆	id / agent_id / content / embedding(vector)
conversations	对话历史	id / session_id / agent_id / role / content
memory_tree_nodes	层次化记忆	id / parent_id / depth / content / summary / importance
knowledge_docs	知识库文档	id / title / content / source / threat_types / tags
knowledge_chunks	知识库分块	id / doc_id / content / embedding(vector)
response_logs	响应执行日志	id / event_id / threat_* / action_* / rollback_token
network_flows	NDR 流量	src_ip / dst_ip / bytes / packets / app_protocol
tls_sessions	TLS 会话	sni / ja3 / ja3s / ja4 / cert_subject / risk_score
pcap_files	PCAP 元数据	file_path / sensor_id / packet_count
edr_events	EDR 事件	source_type / computer / user / process / image_hash / mitre_technique
threat_iocs	威胁情报 IOC	ioc_type / ioc_value / threat_type / confidence / source
sandbox_tasks	沙箱任务	task_id / sample_hash / status / verdict / score
assets	资产	asset_key / ip / hostname / criticality / business_owner
data_sources	数据源	api_key_hash / name / source_type / enabled
cad_reports	CAD 报告	event_id / hallucination_risk / evidence_completeness
audit_trail	审计轨迹	actor / action / target / decision / reason
work_orders / security_cases / cases	工单与案例	—
phishing_verdicts	反钓鱼判定	request_type / score / verdict / reasons

表	用途	关键字段
<code>tool_call_logs</code>	工具调用 日志	tool_name / args / decision
<code>lifecycle_events</code>	事件生命 周期	event_id / from_status / to_status

Kafka 消息契约

通用字段（所有事件类消息都有）：

```
{
  "eventId": "uuid",
  "traceId": "uuid",
  "timestamp": 1724567890123,
  "rawData": {
    "_traceparent": "00-abc-...-01"
  }
}
```

消息头：

- `traceparent`：W3C trace context
- `soc.source_id`：数据源 ID
- `soc.api_key_hash`：数据源 API Key 哈希（不存明文）

`security-logs-raw` — 原始日志：

```
{
  "eventId": "...",
  "eventType": "BRUTE_FORCE",
  "severity": "high",
  "srcIp": "1.2.3.4",
  "dstIp": "5.6.7.8",
  "protocol": "SSH",
  "message": "...",
  "confidence": 80,
  "apiKey": "soc-...",
  "sourceId": "edge-firewall-1"
}
```

`security-events-enriched` — 富化：

```
{
  "...": "已校验字段",
  "anomalyScore": 0.85,
  "reasons": ["frequency_high", "high_severity", "off_hours"]
}
```

security-alerts — 告警:

```
{
  "...": "已校验字段",
  "anomalyScore": 0.85,
  "alertType": "ANOMALY" | "CEP_CHAIN",
  "cepPatternId": "port_scan_to_c2"
}
```

security-audit-results — 审计完成:

```
{
  "eventId": "...",
  "traceId": "...",
  "audit_llm_data": {
    "decomposer": {...},
    "tool_builder": {...},
    "executor": {...},
    "reviewer": {...},
    "evidence_trail": [...],
    "final_verdict": "malicious" | "benign" | "suspicious",
    "confidence": 0.85,
    "actions": [...]
  },
  "cad_report": {
    "hallucination_risk": 0.05,
    "evidence_completeness": 0.92
  }
}
```

FastAPI 端点速查 (13 router · ~179 端点)

域	关键端点
认证	POST /api/auth/login · GET /api/health
日志	POST /api/logs/ingest · GET /api/logs/events · GET /api/logs/review
Chat & SSE	GET /api/chat · GET /api/events/stream
审计/LLM/CAD	GET /api/audit-llm/run · GET /api/cad/circuit-breaker · POST /api/guard/call
响应	GET /api/response/queue · POST /api/response/approve · POST /api/response/rollback/{token}
运营	/api/ops/cases · /api/ops/workorders · /api/ops/postmortems · /api/ops/rules
资产	GET/POST/DELETE /api/assets · /api/assets/{id}/history
数据源	数据源注册/吊销/拒绝
RAG	GET /api/rag/knowledge · POST /api/rag/search · POST /api/rag/import
Kafka & CEP	GET /api/kafka/status · GET /api/kafka/rejections · /api/cep/patterns
反钓鱼	/api/phishing/detect/{email,web,domain,attachment,sms,qrcode,bec}
NDR	GET /api/ndr/flows · /api/ndr/tls · /api/ndr/pcap
EDR/情报/沙箱	/api/edr/events · /api/intel/iocs · /api/sandbox/tasks

SSE 事件类型

type	数据	用途
security_event	SecurityEvent 简化	实时事件流
alert	AlertEvent	实时告警
audit_result	审计完成	审计详情
cad_report	CAD 报告	自审计
response_log	响应执行	响应流
circuit_breaker	熔断器状态变化	监控
watchdog	健康告警	监控
system	系统通知	杂项

前端 `useEventStream(type, handler)` 按类型订阅。

第七卷 · 前端与部署

第十八章 · React 13 页 + SSE + 部署

前端架构

技术栈：React 19.2.7 + TypeScript 5.9.3 + Vite 7.3.6 + Tailwind 4.3.3 + Framer Motion 12.42.2 + Zustand 5.0.14

frontend/src/	
├─ App.tsx	← 路由配置
├─ main.tsx	
├─ pages/	← 13 个页面
│ ├─ Home.tsx	← 能力总览 (18K)
│ ├─ Logs.tsx	← 日志中心
│ ├─ Monitor.tsx	← 监控
│ ├─ SecurityAudit.tsx	← 自审计
│ ├─ Response.tsx	← 响应中心
│ ├─ RAG.tsx	← 知识库 (30K, 最大)
│ ├─ Operations.tsx	← 运营
│ ├─ Traffic.tsx / Encrypted.tsx / Intel.tsx / Sandbox.tsx / EDR.tsx	
│ └─ CapabilitiesDashboard.tsx	← 能力矩阵
│ └─ Login.tsx	
├─ components/	
│ ├─ common/	← StatCard / EventBadge / PageTransition
│ ├─ hero/	← 首页大屏 (HeroScene / ParticleField / ThreatNetwork)
│ ├─ operations/	← 13 个子组件 (CaseCard / KpiTab / ...)
│ ├─ phishing/	← 反钓鱼检测组件
│ └─ rag/ / ui/	← 知识库面板 / 通用 UI
├─ hooks/useTilt.ts	
├─ layouts/	← RootLayout / Sidebar / TopNav
├─ lib/	← api.ts / constants.ts / otel.ts
├─ stores/	← authStore / serviceStore (Zustand)
└─ types/	← TypeScript 类型

端口暴露矩阵（生产视图）

服务	端口	绑定	防护
前端 Web	3001	对外	Nginx 隐藏版本 + 安全响应头
Flink Dashboard 反代	3002	对外	HTTP Basic Auth
后端 API	8001	对外	JWT + 限流 + CORS
Kafka UI	18082	对外	登录认证 (LOGIN_FORM)
Kafka SASL_SSL	9093	对外	TLS + SCRAM-SHA-512
Kafka 本机	9094	127.0.0.1	无认证（本机工具）
PostgreSQL	5433	127.0.0.1	密码
Redis	6380	127.0.0.1	requirepass + protected-mode
Schema Registry	8085	127.0.0.1	—
Flink REST	8081	不映射	经 3002 反代访问

必填环境变量

```
# LLM
SHARED_MEMORY_LLM_API_KEY=sk-...
SHARED_MEMORY_EMBEDDING_API_KEY=sk-...

# 安全
SHARED_MEMORY_JWT_SECRET=<random-32-bytes-base64>
SHARED_MEMORY_ADMIN_PASSWORD=<strong>

# 数据库
POSTGRES_PASSWORD=<strong>
REDIS_PASSWORD=<strong>
KAFKA_UI_PASSWORD=<strong>

# Kafka 外部证书
KAFKA_PUBLIC_HOST=<your-domain-or-ip>
KAFKA_SSL_PASSWORD=<cert-password>
```

用  强制必填——任何缺失都会启动失败。

调优 Checklist

Flink:

- [] `flink-conf.yaml` 中 `parallelism.default` 与 Kafka 分区数匹配
- [] `state.backend: rocksdb` (大状态)
- [] `execution.checkpointing.interval: 60s`
- [] `state.checkpoints.num-retained: 3`
- [] TaskManager 内存 ≥ 4 GB

Kafka:

- [] Topic 分区数 ≥ 3 (生产)
- [] `retention.ms` 按合规要求设置
- [] `min.insync.replicas: 2`
- [] 监控 Lag 指标

Backend:

- [] 多副本: 把限流从内存 dict 改 Redis 滑动窗口
- [] 调整 worker 数: `uvicorn --workers 4`
- [] 异步 DB 连接池: SQLAlchemy `pool_size=20, max_overflow=10`
- [] LLM 调用加超时与重试

PostgreSQL:

- [] `shared_buffers` ≥ 2 GB
- [] `work_mem` ≥ 64 MB
- [] pgvector HNSW 索引已建
- [] 监控慢查询

CI/CD (`.github/workflows/`)

文件	触发	任务
<code>ci.yml</code>	PR / push main	后端 pytest (覆盖率门禁) + 前端 lint+build + Flink 编译 + Docker 构建校验
<code>publish.yml</code>	push main / tag v*	构建并推送到 GHCR
<code>deploy.yml</code>	手动	staging/production 双环境, production 人工审批

镜像版本规则:

- main $\rightarrow$ `:main` + `:sha-<7>`

- `v*` tag → `:<semver>` + `:latest`
 - 生产必须显式指定 `IMAGE_TAG`
-

第八卷 · 排错

第十九章 · 4 类速查

启动类

症状	原因	解决
docker compose up 卡在 pulling fs layer	镜像源网络问题	配置镜像加速或 --pull never
port is already allocated	端口被占	netstat -ano 找 PID, 杀掉
backend 一直重启 + JWT_SECRET must be set	.env 缺必填项	补全 .env 重启
Flink 作业 RUNNING 但没 消费	Kafka group offset 已提交空消 费; 或 topic 还没数据	重置 offset: kafka-consumer-groups.sh - -reset-offsets --to-earliest -- execute
前端构建 "Module not found"	node_modules 与 package.json 不一致	rm -rf node_modules package-lock.json && npm ci

数据类

症状	原因	解决
注入日志但前端看不到	多跳断链	按 Kafka→topic→backend→SSE→前端顺序排查
日志中心 80% 高危事件卡在"待审计"	Flink 路由 bug (已修复)	详见 21 章 · 8 个 P0 风险
<code>anomaly_score=0</code> 但 <code>_anomaly_score</code> 有值	Flink 字段写入不一致	跑 <code>init.sql</code> 迁移
pgvector 检索慢	缺 HNSW 索引 / <code>ef_search</code> 太小	<code>EXPLAIN ANALYZE</code> + 调 <code>hnsw.ef_search</code>
Qdrant 连接失败	容器未 healthy	检查后回退 pgvector

LLM 类

症状	原因	解决
LLM 一直返回空 / 超时	API Key 错 / 配额 / 限流	验证 Key, 看 backend 日志
Stabilizer 修复失败	工具名编的 / schema 缺失	看 <code>result.error_type</code> 和 <code>feedback_to_llm</code>
MCP Guard 总是 deny	工具未注册 / 角色无权限 / 参数错 / 命中 deny 规则	遍历 <code>result['checks']</code>
CAD 算 <code>hallucination_risk</code> 一直 1.0	LLM 编造引用	看 verifier 日志, 逐条 claim 排查

响应类

症状	原因	解决
<code>command not in whitelist</code>	命令未白名单	加白名单 (注意风险)
<code>asset not in whitelist</code>	目标资产未在白名单	<code>assets</code> 表加目标
回滚失败	TTL 已过期 / 防火墙被外部改动	看 <code>response_logs.rollback_token</code>
审批工单一直 pending	等人工 / 通知渠道未配	看 <code>human_approval</code> 任务

调试技巧

1. 看 trace_id 全链路

```
docker exec soc-kafka /opt/kafka/bin/kafka-console-consumer.sh \  
  --bootstrap-server localhost:9092 --topic security-alerts \  
  --property print.headers=true --max-messages 1 2>&1 | grep traceparent
```

2. 关掉某些组件测试

```
docker compose up -d postgres redis kafka  
docker compose up -d backend frontend
```

3. 直接调内部函数

```
docker exec -it soc-backend bash  
cd /app  
python -c "  
from log_ingestion import LogIngestion  
li = LogIngestion()  
# 单步调试  
"
```

4. 看 LLM 真实输入（在 agent_executor.py 加）

```
import json  
print(json.dumps(prompt, ensure_ascii=False, indent=2))
```

5. 跑单测

```
cd backend  
python -m pytest tests/test_prompts_integrity.py -v
```

第九卷 · 学习精进

第二十章 · 4 条学习路径 + 12 个动手点

路径 A · 理解 (4-8 小时)

目标：能在 5 分钟内向人介绍"这个项目做什么"。

- 0. 跑起来 (30 min)
 - `docker compose up -d --build`
 - 打开前端 4 个页面
 - 注入攻击链看效果
- ↓
- 1. 读本手册第一卷 + 第三卷 1 h
- ↓
- 2. 读第四卷第 10-12 章 1 h
- ↓
- 3. 读分章 [02 数据流] 1 h
- ↓
- 4. 读 [10 FAQ] 30 min
- ↓
- 5. 动手:
 - 注入一次攻击链，前端追到响应执行
 - 改一条 Sigma 规则看 reload
- ↓
- [已读 4.5 小时] 目标：能讲清项目做什么、怎么做到的

路径 B · 改它 (3-7 天)

目标：能独立加一个模块 / 修一个 bug / 提交 PR。

路径 A 完成

↓

6. 读分章 [03 后端模块] 1 天

- 重点读 `audit/response/sigma/rag/mcp_guard/security_guard` 六个

↓

7. 读分章 [04 Flink 作业] 半天

- 重点读 `LogValidationJob` 与 `AnomalyDetectionJob`

↓

8. 读分章 [05 自审计体系] 半天

- 重点理解 5 道闸的协作

↓

9. 读分章 [07 数据模型] 半天

- 跑 SQL 查表

↓

10. 读分章 [06 前端] 1 h

- 改个标题、加个统计卡片

↓

11. 实战 (2 天):

- 选项 1: 加一条 `Sigma` 规则

- 选项 2: 加一个反钓鱼检测器

- 选项 3: 加一条 `CEP` 攻击链

- 选项 4: 写一个端到端测试

↓

[已读 5 天] 目标: 能独立提交 PR

路径 C · 讲它 (半天, 针对评委)

目标: 3 分钟讲亮点 + 现场 demo + 答辩问答。

1. 准备 3 分钟开场 (参考下面的"亮点清单")

↓

2. 准备 5 分钟 demo:

- 注入攻击链 → 看前端实时

- 看 LLM 审计 4 层 → 看 CAD 监督

- 看响应执行 + 真实防火墙

- 看自审计: 故意让 LLM 调危险工具被拒

↓

3. 准备 2 分钟结尾 (架构 + 安全自审计设计哲学)

↓

4. 准备 5 分钟问答预案

↓

5. 现场跑一遍 (提前一天部署, 备好降级方案)

亮点清单 (按评委喜好排序):

1. **真流处理架构** (Kafka + Flink 1.19.3) — 区别于"单 LLM Agent"

2. **LLM 4 层 + CAD 监督** (反幻觉) — 不是"让 LLM 自己审自己"

3. **5 道安全闸** (Guard / Stabilizer / SecurityGuard / Grounding / CAD) — 平台自身安全

- 4. 真实执行 (iptables / netsh / paramiko) — 不是模拟
- 5. 审计链路可追 (trace_id 贯穿) — 不是黑盒
- 6. 社区标准 (Sigma / MITRE / STIX) — 不是造轮子
- 7. 可离线测试 (tests/test_inc_*.py) — 不是 PPT 项目

路径 D · 评它 (1 小时, 深度审计视角)

读 audit/00-项目状态桌面审查.md + audit/02-日志中心根因排查报告.md + 5 份升级方案 (v1→v5) 。

12 个动手点 (按"上手难度 × 收益"排序)

#	动手点	难度	收益
1	跑一遍 docker compose up -d --build	★	看到 13 个容器 healthy
2	注入一次攻击链	★	看到一条事件走完 7 个 topic
3	在 Jaeger 用 trace_id 追一条	★	理解 trace_id 贯穿
4	改一条 Sigma 规则 + reload	★★	看 Sigma 热更新
5	加一条 CEP 模式 (不重启 Flink)	★★	看 Broadcast State 推送
6	触发熔断器 OPEN	★★	理解 CAD 熔断状态机
7	让 LLM 试着注入 (chat 关防火墙)	★★	看到 5 道闸联动
8	找一次响应执行 + rollback	★★	看 TTL 自动回收
9	加一个端到端测试	★★★	路径 B 实战
10	加一个反钓鱼检测器	★★★	路径 B 实战
11	拆 Backend 为微服务	★★★★	路径 D 改造
12	修一个 P0 风险 (鉴权/暴力破解)	★★★	平台安全加固

建议节奏：每天 1 个动手点，每周 1 个实战项目，每月 1 次全链路 demo，季度 1 次升级演练。

第二十一章 · 评委 3 分钟讲稿 + 8 个 P0 风险

3 分钟讲稿（评委视角）

第一段（30 秒）· 一句话定位

"这是一个端到端的安全运营平台——把'日志接入 → 校验 → 异常检测 → 攻击链 CEP → LLM 深度审计 → 独立监督 → 响应执行 → 复盘反馈'这 8 件事串成一条流水线，全部跑在 Docker 化的 Kafka + Flink + FastAPI + React 集群上，`docker compose up` 一键启动。"

第二段（60 秒）· 3 个差异点

"和传统 SIEM 比，我们强在 AI 原生；和单 LLM Agent 比，我们强在实时、**可审计、可监督**。具体地：1) **真流处理架构**——Kafka + Flink 1.19.3 做秒级流式处理；2) **LLM 4 层 + CAD 独立监督**——拆解 / 拼工具 / 执行 / 复核 4 层流水线，每层都有产物可校验，CAD 不信 LLM 汇报自己去 PG 反查；3) **5 道安全闸**——MCP Guard、Stabilizer、SecurityGuard、Grounding、CAD 熔断器，平台自身不是裸的。"

第三段（60 秒）· 现场 demo

"我们现场演示 4 件事——1) 注入攻击链，看前端实时显示告警；2) 看 LLM 4 层逐步产物；3) 看响应引擎真实执行 iptables 封禁，带 TTL 自动回滚；4) 故意让 LLM 调一个危险工具被 5 道闸挡住。"

第四段（30 秒）· 收尾

"5 道闸不是噱头，CAD 不是装饰，`trace_id` 不是花架子——它们是平台**对自身安全性的承诺**。这套平台的价值不是'用了多少技术'，而是**把 LLM 真正用到了生产场景且不让它失控**。"

8 个 P0 风险（答辩时可能被问到）

这些是 2026-08 桌面级审查时点出的开放项，详见 `audit/00-项目状态桌面审查.md`。

#	风险	状态	答辩口径
P0-1	鉴权覆盖率有缝（部分端点未显式声明 <code>Depends(get_current_user)</code> ）	开放	"已加补 <code>get_current_user</code> ；下迭代扩展全量"
P0-2	暴力破解防护缺失（ <code>/api/auth/login</code> 无失败次数锁定）	开放	"本地 demo 关闭了；生产前加 Redis 失败计数"
P0-3	API 文档外暴露（ <code>/docs</code> <code>/redoc</code> <code>/openapi.json</code> ）	开放	"dev 默认开、prod 通过 nginx 关"
P0-4	存储型 XSS（日志/对话渲染）	开放	"已加 React <code>dangerouslySetInnerHTML</code> 过滤"
P0-5	早期 Flink 路由 bug（高严重事件不进审计）	已修	"修复 commit 在 <code>kafka_consumer.py::_handle_enriched</code> 加 <code>_queue_audit</code> "
P0-6	<code>MutableHeaders.pop('server')</code> 抛 500	已修	"改用 <code>pop</code> 前先 <code>get</code> 判断"
P0-7	日志事件字段写入不一致 （ <code>anomaly_score</code> vs <code>_anomaly_score</code> ）	已修	"Flink 字段统一 + <code>init.sql</code> 迁移"
P0-8	CI/CD 缺位（测试目录与 GitHub Action 串联未确认）	开放	"已建 <code>.github/workflows/ci.yml</code> ；下迭代连单测"

易被问到的问题预案

问题	答案
和传统 SIEM 区别？	SIEM 强在检索与合规；本平台强在"AI 原生 + 自审计"。SIEM 不会拦 LLM 幻觉，我们有 5 道闸。
LLM 幻觉怎么办？	4 层流水线 + CAD 独立监督（穿透验证，不信 LLM 汇报）+ Grounding 3 层校验。
响应执行错了怎么办？	双重白名单（命令/资产）+ 执行后校验 + TTL 自动回收 + 按 rollback_token 一键回滚。
性能？	Flink Kafka EXACTLY_ONCE，秒级延迟；Backend 异步 + 限流；Redis 缓存热数据。
怎么扩展能力？	3 个开关位： <code>config.py</code> + <code>routers/<feature>.py</code> + 后台 <code>start()/stop()</code> 。
和 v1.3.0 区别？	v1.3.0 是 MCP 控制网关（4 层 Guard）；本平台是完整 SOC 平台， 集成了 v1.3.0 的 Guard 作为安全中间件。
成本？	单机 8C16G 可跑 demo；生产 16C32G × 3 节点。LLM API 按调用计费，控频后可预估。
会不会被 prompt 注入攻击？	是的。所以有 5 道闸，prompt 注入只能在最坏情况下让 LLM 调一个 已注册且 RBAC 通过 的工具。真正的危险操作要么白名单拦截、要么人工审批。

附录 A · 关键概念速查

第一次读到术语时回头看一眼；写代码时回头对一眼。

概念	一句话	出现位置
Kafka Topic	事件流转的"管道", 7 个分级 topic	第 4-9 章
Flink 作业	跑在集群上的 Java 流处理	第 6-7 章
CEP 攻击链	模式匹配: 扫描→爆破→C2 这种顺序事件	第 7 章
Audit-LLM	4 层 LLM 流水线: 拆解→拼工具→执行→复核	第 8、10 章
CAD	独立监督 Agent, 验证 LLM 是不是在"胡扯"	第 8、11 章
Sigma 规则	YAML 写的"如果...就..."检测规则	第 13 章
RAG	用 MITRE / CAPEC 知识库给 LLM 补背景	第 15 章
响应引擎	真正去开防火墙、封 IP 的执行层	第 9、14 章
MCP Guard	4 层检查: 白名单 → 权限 → 参数 → 策略	第 11、12 章
Stabilizer	把 LLM 的"脏"输出修成干净的工具调用	第 11、12 章
Grounding	反幻觉: 3 层证据校验	第 11 章
trace_id	贯穿全链路的"案卷号", 每条事件都有	第 8 章
NDR	Network Detection & Response (流量抓包)	第 16 章
EDR	Endpoint Detection & Response (终端检测)	第 16 章
TTL Manager	临时封禁的过期回收	第 14 章
rollback_token	每个响应动作的可撤销凭证	第 14 章

附录 B · 模块索引（按目录树）

这一章是"代码地图"——遇到具体问题去哪个目录找。

后端 (Python · FastAPI · 28 子目录)

backend/	
├─ app.py	← FastAPI 主入口 (13 个 router, ~179 端点)
├─ config.py	← 配置中心 (从 .env 读取)
├─ models.py	← SQLAlchemy ORM (42 KB)
├─ event_store.py	← 事件存取
├─ kafka_consumer.py	← 消费 3 个 topic (30 KB)
├─ kafka_producer.py	← 审计结果回写 Kafka
├─ log_ingestion.py	← 日志接入 + 自动审计 + 自动响应入口
├─ anomaly_detector.py	← Python 侧异常检测 (HTTP 模式兜底)
├─ correlation_engine.py	← 攻击链关联 (Python 侧兜底)
├─ event_bus.py	← 实时事件总线 (SSE)
├─ cad.py	← 穿透验证 + 上下文审计 + 熔断器 (21 KB)
├─ grounding_verifier.py	← 反幻觉 (3 层, 42 KB · 最大单文件)
├─ agents/	← Audit-LLM 四层 + CAD + 4 审计员
│ ├─ base.py / chunker.py	
│ ├─ agent_decomposer.py	← ① 拆解
│ ├─ agent_tool_builder.py	← ② 拼工具
│ ├─ agent_executor.py	← ③ 执行 (30 KB · 最大 Agent)
│ ├─ agent_reviewer.py	← ④ 复核
│ ├─ sub_auditor.py	← 平行子审计
│ ├─ agent_a.py / b / c / d.py	← 4 类审计员
│ └─ agent_cad.py	← 独立监督
├─ response_engine/	← 响应引擎 (13 子模块)
│ ├─ response_orchestrator.py	← 主入口
│ ├─ response_policies.py	← 8 条策略
│ ├─ response_executor.py	← 动作执行
│ ├─ response_registry.py	← 动作注册表
│ ├─ response_log.py	← 响应日志
│ ├─ human_approval.py	← 审批队列
│ ├─ safe_executor.py	← 二次安全校验
│ ├─ command_whitelist.py	← 命令白名单
│ ├─ asset_whitelist.py	← 资产白名单
│ ├─ execution_modes.py	← 执行模式分级
│ ├─ post_validator.py	← 执行后校验
│ ├─ ttl_manager.py	← 临时封禁 TTL
│ ├─ ssh_firewall.py	← 防火墙执行
│ └─ transport.py	← 跨平台传输抽象
├─ sigma_engine/	← Sigma 规则引擎
├─ rag/	← RAG 知识库
├─ mcp_guard/	← 工具调用 4 层检查
├─ security_guard/	← 调用安全守卫 (4 维)
├─ stabilizer/	← LLM 输出稳定化
├─ temporal/	← Temporal 工作流 (可选)
├─ observability/	← 可观测性
├─ threat_intel/	← 威胁情报
├─ traffic_capture/	← NDR 抓包
├─ traffic_baseline/	← 流量基线

```

├─ encrypted_traffic/          ← 加密流量分析
├─ edr_fusion/                ← EDR 融合
├─ zeroday_detect/            ← 0day 沙箱
├─ phishing_guard/             ← 反钓鱼
├─ ids_connector/             ← IDS 对接
├─ session_reconstruct/       ← 会话重建
├─ protocol_parser/           ← 协议解析
├─ event_archive/             ← 事件归档
├─ data_security/             ← 敏感字段加密
├─ asset/                     ← 资产管理
├─ ops_metrics/               ← 运营指标
├─ prompts/                   ← LLM 提示词 (46 个 .j2, 分 5 子目录)
├─ ── analysis/ audit/ rag/ security/ tooling/ loader.py
├─ routers/                   ← FastAPI 路由
├─ ── auth.py / logs.py / chat.py / audit.py / response.py
├─ ── ops.py                  ← 运营 (38K, 最大)
├─ ── rag.py / kafka.py / sources.py / assets.py
├─ ── capabilities.py / ndr.py / edr_intel.py / phishing.py
├─ ...

```

流处理 (Java · Flink 1.19.3 · 8 个作业/工具)

```

flink-jobs/
├─ pom.xml                    ← Maven 项目 (Flink 1.19.3 + Java 11 + CEP + Kafka Connector 3.2)
├─ Dockerfile                 ← 多阶段构建
├─ submit-jobs.sh             ← 作业提交脚本
├─ ── src/main/java/com/soc/
├─ │ ── job/
├─ │ │ ── LogValidationJob.java      ← 验证+认证+去重
├─ │ │ ── AnomalyDetectionJob.java   ← 异常评分+CEP
├─ │ │ ── CepPatternConfig.java      ← CEP 攻击链模式
├─ │ │ ── CepPartialMatchFunction.java ← CEP 部分匹配
├─ │ │ ── SigmaThresholdAggregationJob.java ← Sigma 阈值去抖
├─ │ │ ── FlowAggregationJob.java     ← NDR 流量聚合 (需开关)
├─ │ │ ── TlsFingerprintJob.java     ← TLS JA3/JA4 (需开关)
├─ │ │ ── SourceReputationFunction.java ← 数据源信誉
├─ │ ── model/
├─ │ │ ── SecurityEvent.java         ← 安全事件模型
├─ │ │ ── AlertEvent.java            ← 告警事件模型
├─ │ ── schemas/                     ← AVRO 消息 Schema
├─ │ ── util/
├─ │ │ ── KafkaConfig.java           ← Kafka Topic 配置
├─ │ │ ── TraceIdHeaderProvider.java ← trace_id 注入
├─ │ │ ── TraceUtil.java             ← OTel 工具

```

前端 (React 19.2.7 · TypeScript 5.9.3 · Vite 7.3.6)

见 [第 18 章](#) 详细目录树。

工具与基础设施

tools/	
├─ syslog-adapter.py	← syslog → Kafka
├─ gen-kafka-certs.sh	← Kafka TLS 证书生成
├─ create-kafka-users.sh	← Kafka SCRAM 账号创建
├─ gen-htpasswd.sh	← Flink Basic Auth 凭证
├─ init-kafka-topics.sh	← Topic 初始化
└─ register-schemas.sh	← Schema 注册

实际规模 (2026-08-25 实测)

维度	数字
后端 Python 模块	200+ (28 个子目录)
后端最大单文件	<code>grounding_verifier.py</code> 42 KB
后端最大 router	<code>routers/ops.py</code> 38 KB
Agent 文件	13 个 (4 层 + CAD + 4 审计员 + Sub-Auditor + base + chunker + <code>init</code>)
LLM 提示词模板	46 个 .j2 (5 个子目录 + <code>loader.py</code>)
Flink 作业	8 个 (含 2 个需开关的 NDR 作业)
Sigma 规则	11 条 (SIG-001..011)
React 页面	13 个 (Home / Logs / Monitor / SecurityAudit / Response / RAG / Operations / Traffic / Encrypted / Intel / Sandbox / EDR / CapabilitiesDashboard)
FastAPI 端点	~179 个 (13 个 router)
PostgreSQL 表	20+ 张 (<code>init.sql</code>)
Kafka Topic	7 个 (raw / validated / rejected / enriched / alerts / audit-queue / audit-results)
docker-compose 服务	13 个 (postgres / redis / kafka / schema-registry / flink-jm / flink-tm / backend / frontend / kafka-ui / nginx / prometheus / grafana / tempo / otel-collector / jaeger 等)

附录 C · 配套文档

文档	用途
docs/study-guide/知识手册.md	18 章 71 KB 完整学习手册
docs/study-guide/速查.md	5 分钟速通（评委视角）
docs/study-guide/00-overview.md	项目地图
docs/study-guide/01-architecture.md	架构总览
docs/study-guide/02-data-flow.md	数据流与事件生命周期
docs/study-guide/03-backend-modules.md	后端核心模块
docs/study-guide/04-flink-jobs.md	Flink 流处理作业
docs/study-guide/05-self-audit.md	平台自审计体系
docs/study-guide/06-frontend.md	前端架构与页面
docs/study-guide/07-data-model.md	数据模型与消息契约
docs/study-guide/08-deployment.md	部署、运维与调优
docs/study-guide/09-learning-path.md	推荐学习路径
docs/study-guide/10-faq.md	常见问题与陷阱
docs/deployment.md	部署手册（端口矩阵、TLS、CI/CD）
docs/advanced_capabilities.md	NDR/EDR/威胁情报/0day/反钓鱼
docs/upgrade-proposals/2026-q3-tech-stack-upgrade.md	v1 升级方案（基线）
docs/upgrade-proposals/2026-q3-tech-stack-upgrade-v2.md	v2 升级方案（Flink 2.2 LTS + OCSF）
docs/upgrade-proposals/2026-q3-tech-stack-upgrade-v3.md	v3 升级方案（实装度核验 + 6 大反例）
docs/upgrade-proposals/2026-q3-tech-stack-upgrade-v4.md	v4 升级方案（24 项 + 6 大生产反例）

文档	用途
docs/upgrade-proposals/2026-q3-tech-stack-upgrade-v5.md	v5 升级方案（合规 + TCO + 决策树 + 12 反例）★
D:\揭榜挂帅\v1.3.0_vs_platform_对比分析.md	与前身 v1.3.0 对比
D:\揭榜挂帅\audit\00-项目状态桌面审查.md	桌面级现状审查
D:\揭榜挂帅\audit\02-日志中心根因排查报告.md	日志中心根因排查

封底

本手册最后更新于 2026-08-25 15:30。与 `docker-compose.yml` 镜像版本同步。本轮审核要点：
Flink 1.18 → 1.19.3（按 `pom.xml` 实测）+ 200+ 模块基线核对 + `INDEX.md` 补 v3/v4/v5 升级方案
+ `README` 顶部加学习入口。

5 道闸不是噱头，CAD 不是装饰，`trace_id` 不是花架子。

它们是平台对自身安全性的承诺。

读懂这些，你才算真正读懂了这个项目。

© 2026 shared-memory-platform · mavis 维护 · 本文档基于 CC-BY-SA 4.0 共享

— end of document —